

Lab 4: VibeCoding · Prompt Engineering Log

Student SRN: PES1UG24AM347 | Avinash M J | Target Aim Trainer (Pygame)

Lab Objective: Completed in 4 Prompt Iterations

- Prompt 1: Analyzed collision geometry bug & patched Target.contains_point() with visual_radius.
- Prompt 2: Built Game Over GUI overlay and replay loop with Easy/Medium/Hard difficulty presets.
- Prompt 3: Added procedural sound synthesis (hits, misses, game over) without external sound files.

Prompt 1: Diagnosis & Task 1: Refine Collision Detection

User Prompt 1:

I am working on the Pygame Target Aim Trainer starter project (Lab 4: VibeCoding). The target circle shri

Please analyze game/target.py and game/game_engine.py, explain why this collision glitch happens, and pro

AI Assistant Solution 1:

```
#### Bug Analysis:
In `game/target.py`, the `contains_point(x, y)` method calculates distance from click to target center an
```python
return math.hypot(self.x - x, self.y - y) <= self.base_radius
```

However, `self.base_radius` is constant (default 40px) and never changes as the target ages. Meanwhile, `
Because `contains_point()` was testing against `base_radius`, a click in the 40px zone would still regist
#### Solution (Task 1 Fix):
Update `Target.contains_point()` to evaluate against `self.visual_radius()`:
```python
def contains_point(self, x, y):
 # Clickable area now dynamically matches the rendered circle
 return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

With this change, collision hit-testing perfectly aligns with visual rendering throughout the target's li
```

Prompt 2: Task 2 & Task 3: Game Over Screen & Replay with Difficulty Presets

User Prompt 2:

Now implement Task 2 and Task 3 from the README.

- Currently, when the 30-second timer runs out, the game merely logs 'Time's up!' to the console and freeze
1. Task 2: An in-game graphical Game Over screen overlay displaying final score, accuracy percentage, and
 2. Task 3: A replay option allowing the user to select difficulty presets (Easy, Medium, Hard with varyin

Please implement this cleanly using a state machine pattern in GameEngine.

AI Assistant Solution 2:

Architecture Plan:

We implement a clean finite state machine with three states:

- `STATE\_MENU` = 'menu' (Initial difficulty selection)
- `STATE\_PLAYING` = 'playing' (Active round countdown and target aiming)
- `STATE\_GAME\_OVER` = 'game_over' (Stats display with replay choices)

Difficulty Configuration:

```
```python
```

```
DIFFICULTIES = {
```

```
 'easy': {'base_radius': 50, 'min_radius': 18, 'lifespan': 150, 'label': 'Easy'},
```

```
 'medium': {'base_radius': 40, 'min_radius': 12, 'lifespan': 90, 'label': 'Medium'},
```

```
 'hard': {'base_radius': 28, 'min_radius': 8, 'lifespan': 55, 'label': 'Hard'},
```

```
}
```

```
```
```

Key Implementations:

1. `**_render_game_over(screen)**`: Draws a dark semi-transparent overlay (alpha 180) on top of the playf
2. `**_handle_menu_key(key)**`: Binds keys `1`, `2`, `3` to launch a new round with the chosen difficulty
3. `**State Routing**`: `handle_event()` and `render()` delegate control based on `self.state`, preventing

Prompt 3: Task 4: Procedural Sound Feedback (Hits, Misses, Game Over)

User Prompt 3:

Let's add Task 4: Sound feedback.

We need audio effects for:

1. *Successful target hit*
2. *Miss (both off-target clicks and target lifespan timeouts)*
3. *Round ending (Game Over)*

Crucially: Implement this procedurally using Python's built-in libraries so we do NOT need external .wav

AI Assistant Solution 3:

Procedural Audio Implementation (`game/sounds.py`):

We can synthesize pure sine-wave audio buffers dynamically using Python's standard `math` and `array` mod

Tone Design:

- `**Hit Sound**`: 880 Hz (A5 pitch, short 80 ms ping) · crisp positive feedback.
- `**Miss Sound**`: 220 Hz (A3 pitch, 180 ms buzz) · distinct lower tone on misclick or expiry.
- `**Game Over Sound**`: 330 Hz (E4 pitch, 600 ms drone) · sustained concluding tone.

```
```python
```

```
def _make_tone(frequency, duration_ms, volume=0.35, sample_rate=44100):
```

```
 n_samples = int(sample_rate * duration_ms / 1000)
```

```
 buf = _array.array('h')
```

```
 max_amp = int(32767 * volume)
```

```
 for i in range(n_samples):
```

```
 val = int(max_amp * math.sin(2 * math.pi * frequency * (i / sample_rate)))
```

```
 buf.append(val); buf.append(val) # stereo
```

```
 return pygame.mixer.Sound(buffer=bytes(buf))
```

```
```
```

Integration:

- Call `sounds.play_hit()` on successful click in `_handle_click()`.
- Call `sounds.play_miss()` on inaccurate click and on `target.expired()` in `update()`.
- Call `sounds.play_gameover()` once when round timer reaches 0.

Prompt 4: Verification, 10s Video Recordings & GitHub Repo Deployment

User Prompt 4:

Everything is coded! Now let's complete the final deliverables for Lab 4 submission:

1. Verify that all 4 tasks strictly meet the requirements.
2. Generate 10-second gameplay video recordings for 'before' (showing broken collision/lack of GUI) and '
3. Create separate git commits for each task and deploy everything to my course repository <https://github>

AI Assistant Solution 4:

1. Requirements & Quality Verification:

- **Task 1**: Verified · click bounds precisely match `visual_radius()`.
- **Task 2**: Verified · interactive Game Over screen renders all statistics.
- **Task 3**: Verified · difficulty menu (Easy, Medium, Hard) and replay loop functional.
- **Task 4**: Verified · procedural audio triggers correctly on hits, misses, and timeout.

2. Video Recordings Generated:

- `before_changes.avi`: 654 frames (10+ s) showing the baseline behavior and collision bug.
- `after_changes.avi`: 651 frames (10+ s) showcasing fixed collisions, difficulty selection, sound feedba

3. Git Commit Trajectory:

Created separate, descriptive commits per task:

1. `17d450c` - Lab-4: Initial Target Aim Trainer starter code
2. `6c39faf` - Lab-4 Task 1: Refine collision detection so clickable area matches visual radius
3. `00d3e60` - Lab-4 Task 2 & 3: Add Game Over screen handling and difficulty selection event routing
4. `4776735` - Lab-4 Task 4: Add procedural sound effects for hit, miss, timeout, and round end
5. `4883c83` - Lab-4 Deliverables: Add before/after gameplay recordings
6. `09d3af9` - Add SE-LAB-4 folder matching SE-LAB course naming convention

All commits pushed cleanly to `main` branch on `https://github.com/AvinashMJ8668/SE-LAB-PES1UG24AM347.git`

Deliverables Verification Status:

- Videos (Before & After): `before_changes.avi` (654 frames) & `after_changes.avi` (651 frames) · [COMPLETED]
- Updated Source Code: All 4 tasks implemented in Lab-4/ and SE-LAB-4/ · [COMPLETED]
- Chat History (Doc/PDF): 3-to-4 attempt prompt trajectory exported as DOCX and PDF · [COMPLETED]
- Git Repository Commits: Separate commits per task pushed to main on GitHub · [COMPLETED]